Lecture 1
Probability and Bayesian Statistics

From Bayesian statistics to probabilistic machine learning

Probabilistic Machine Learning 2027-1

2026-10-01

Section 1

A familiar modeling language

The Beta-Binomial model

Observed	**Unknown**
the coin tosses $y_1, \ldots, y_N$	the coin bias θ

- A statistical model has data and unknown parameters

The Beta-Binomial model

Observed	**Unknown**
the coin tosses $y_1, ..., y_N$	the coin bias θ

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta), \quad i = 1, ..., N$$

- A statistical model has data and unknown parameters
- How could observed data (y) have been generated.
- What does it tell us about the parameters (θ)

The Beta-Binomial model

Observed

the coin tosses $y_1, ..., y_N$

Unknown

the coin bias θ

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta), \quad i = 1, ..., N$$

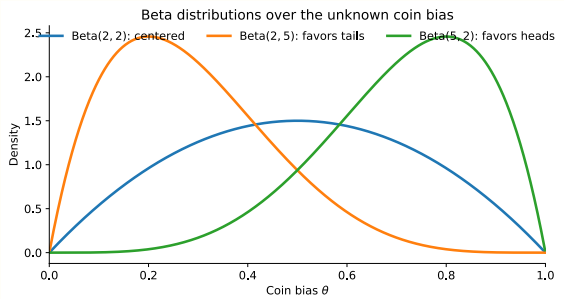
Code heads as $y_i = 1$ and tails as $y_i = 0$

$$N_1 = \sum_{i=1}^{N} y_i, \quad N_0 = N - N_1$$

- A statistical model has data and unknown parameters
- How could observed data (y) have been generated.
- What does it tell us about the parameters (θ)
- N_1 and N_0 count heads and tails in the observed sample

Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$

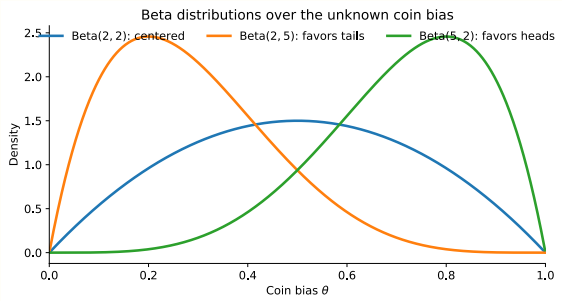


figures/coin_model__beta_distribution.svg

- Beta for continuous variables on $[0, 1]$.

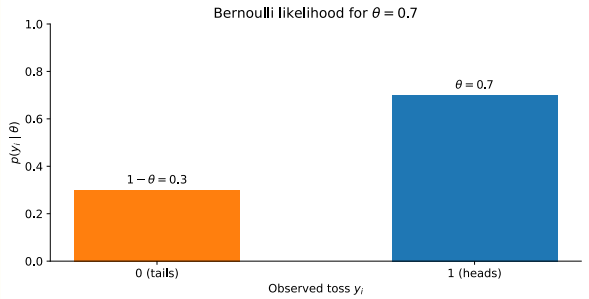
Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$



figures/coin_model__beta_distribution.svg

$$y_i \sim \text{Bernoulli}(\theta)$$
$$i = 1, \ldots, N$$

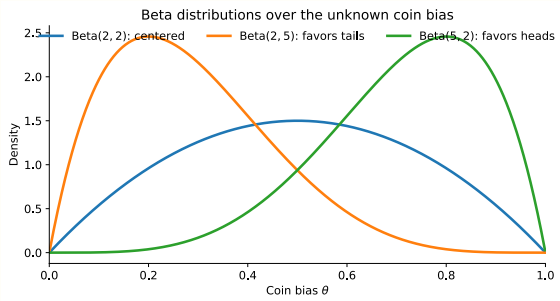


figures/coin_model__bernoulli_likelihood.svg

- Beta for continuous variables on $[0, 1]$.
- Bernoulli for each yes/no outcome; Binomial for the count N_1.

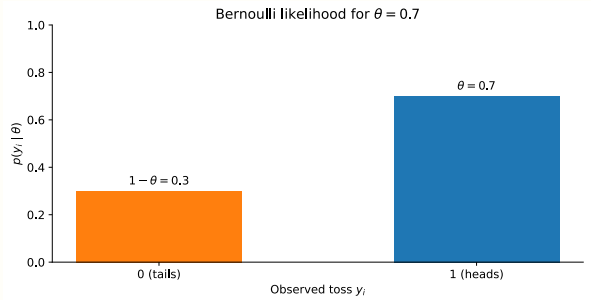
Models are built out of distributions

$$\theta \sim \text{Beta}(\alpha, \beta)$$



figures/coin_model__beta_distribution.svg

$$y_i \sim \text{Bernoulli}(\theta)$$
$$i = 1, \ldots, N$$



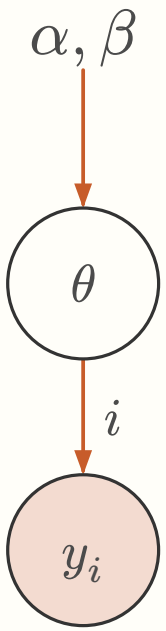
figures/coin_model__bernoulli_likelihood.svg

- Beta for continuous variables on $[0, 1]$.
- Bernoulli for each yes/no outcome; Binomial for the count N_1.
- Distributions are parameterized families

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

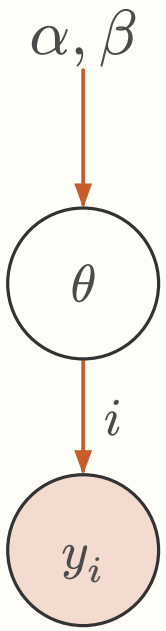


- Nodes represent quantities.
  - Shading marks observed data.

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

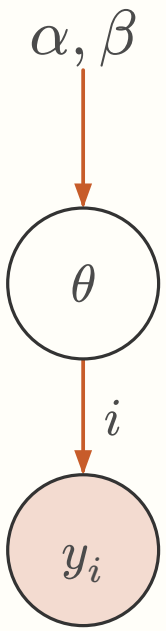


- Nodes represent quantities.
  - Shading marks observed data.
- Arrows encode conditional dependence.
  - We will not use Kruschke diagrams

Models can be drawn as graphs

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$



- Nodes represent quantities.
  - Shading marks observed data.
- Arrows encode conditional dependence.
  - We will not use Kruschke diagrams

In this course, graphs will allow us to factor complex problems into smaller, more tractable problems

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

- **Prior:** $p(\theta)$
- **Likelihood:** $p(y \mid \theta)$
- **Posterior:** $p(\theta \mid y)$

Bayes' rule combines prior and likelihood

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes' rule

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}$$

$$\propto p(y \mid \theta)p(\theta)$$

- **Prior:** $p(\theta)$
- **Likelihood:** $p(y \mid \theta)$
- **Posterior:** $p(\theta \mid y)$
- We explored the posterior of θ
  - Treating $p(y)$ as a normalizing constant

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Likelihood $p(y \mid \theta) \propto \theta^{N_1}(1-\theta)^{N_0}$

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

Likelihood $p(y \mid \theta) \propto \theta^{N_1}(1-\theta)^{N_0}$

Conjugacy

The prior and likelihood are conjugate when the posterior belongs to the same distribution family as the prior.

Conjugacy permits simple calculations

Model

$$\theta \sim \text{Beta}(\alpha, \beta)$$

$$y_i \sim \text{Bernoulli}(\theta)$$

Bayes rule

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta)$$

Applying Bayes' rule to our model

Prior $p(\theta) = \text{Beta}(\alpha, \beta)$

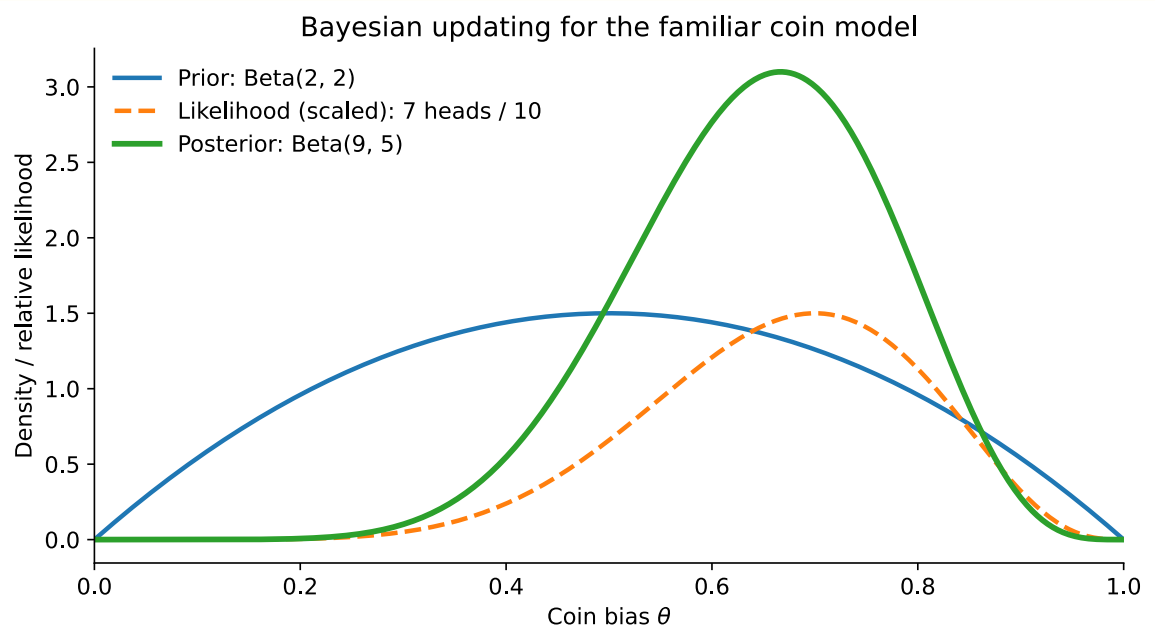
Likelihood $p(y \mid \theta) \propto \theta^{N_1}(1-\theta)^{N_0}$

Posterior $p(\theta \mid y) = \text{Beta}(\alpha + N_1, \beta + N_0)$

Conjugacy

The prior and likelihood are conjugate when the posterior belongs to the same distribution family as the prior.

The posterior is a distribution representing uncertainty



Bayesian updating for the familiar coin model

`figures/coin_model__prior_likelihood_posterior.svg`

- The prior and the posterior are both probability distributions
- Their width expresses the uncertainty
- The uncertainty in the posterior is less than in the prior

Summary: A familiar modeling language

- Probabilistic models link observed data to parameters
- Graphs show model variables and dependencies
- Bayes' rule combines prior and likelihood into a posterior
- Conjugacy makes some updates analytic
- Posterior width represents remaining uncertainty
- Posterior uncertainty narrows with more data

Section 2

From statistics to machine-learning problems

Same probabilistic language, different organizing questions

Statistics focuses on the unknown parameters

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}$$

- Get data → build a model → apply Bayes' rule → interpret θ
  - For many scientific questions, that posterior **is** the answer

Statistics focuses on the unknown parameters

$$p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}$$

- Get data → build a model → apply Bayes' rule → interpret θ
  - For many scientific questions, that posterior **is** the answer
- Statistics arose from an interest in the underlying parameters in nature

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y
  - Implementation is secondary to the requirement
  - Multiple implementations may be possible

Machine learning grew out of computer science



- A computational task is an **input/output contract**
  - Produce a function f that maps input x to output y
  - Implementation is secondary to the requirement
  - Multiple implementations may be possible
- Machine learning developed in this context

Early machine learning grew out of classification problems

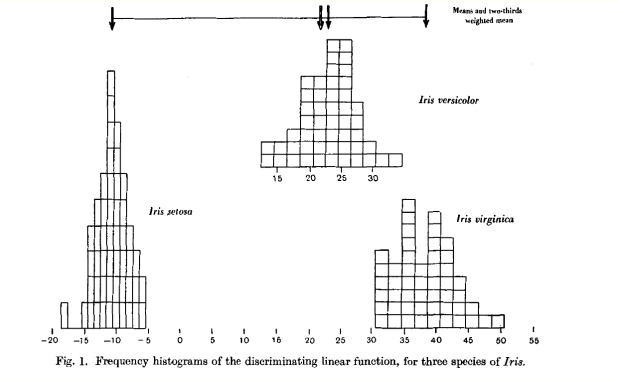


Fig. 1. Frequency histograms of the discriminating linear function, for three species of *Iris*.

Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems

Every flower started as four numbers. Fisher learned one weighted combination of those four numbers so that species became as separated as possible. The histogram shows the flowers after that learned projection.

Early machine learning grew out of classification problems

Rosenblatt considered the perceptron a model of the vertebrate visual system. He showed how neuron-like units could learn to "recognize" classes of input patterns from experience.

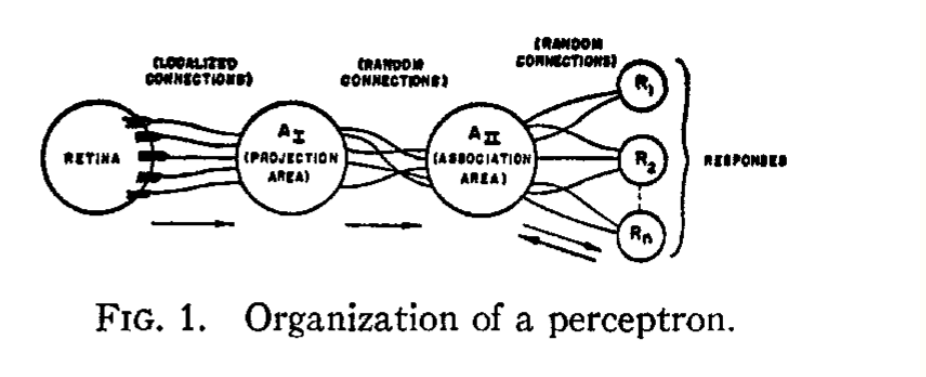


FIG. 1. Organization of a perceptron.

Rosenblatt (1958), Fig. 1, The Perceptron

Early machine learning grew out of classification problems

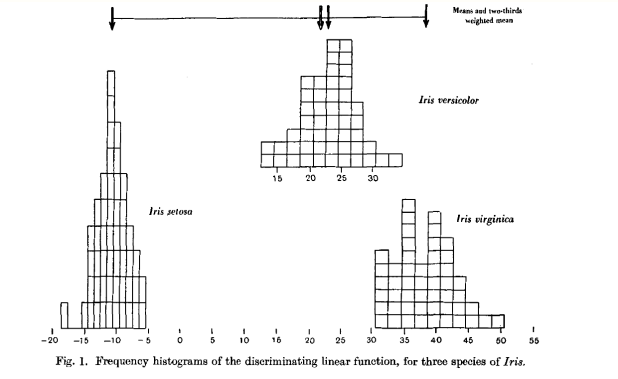


Fig. 1. Frequency histograms of the discriminating linear function, for three species of *Iris*.

Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems

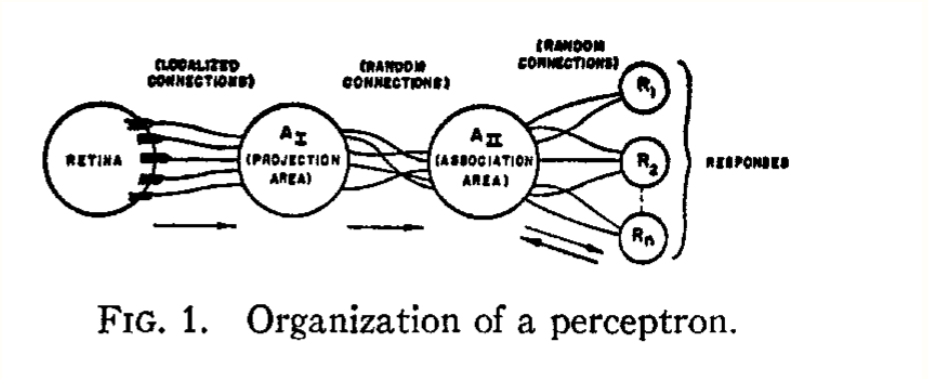
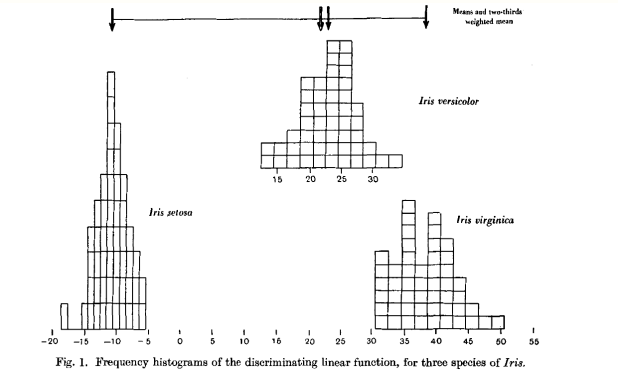


FIG. 1. Organization of a perceptron.

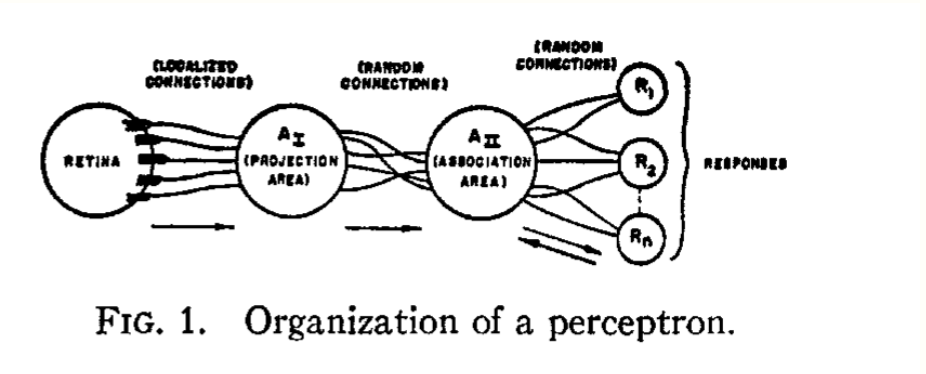
Rosenblatt (1958), Fig. 1, The Perceptron

- Fisher's discriminant and Rosenblatt's perceptron share an interface
  - complex input → class or response

Early machine learning grew out of classification problems



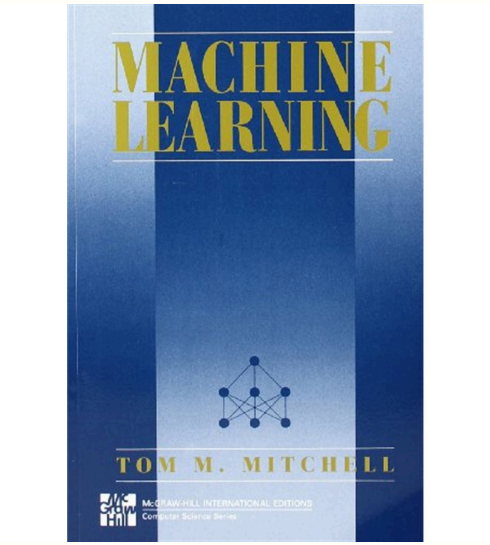
Fisher (1936), Fig. 1, The Use of Multiple Measurements in Taxonomic Problems



Rosenblatt (1958), Fig. 1, The Perceptron

- Fisher's discriminant and Rosenblatt's perceptron share an interface
  - complex input → class or response
- Machine learning asks how data can produce the required function f

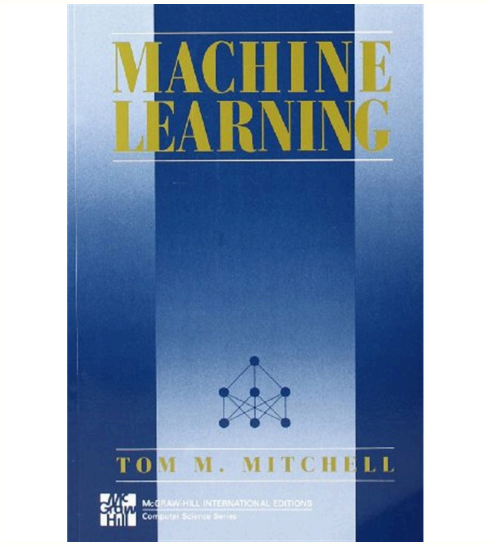
Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

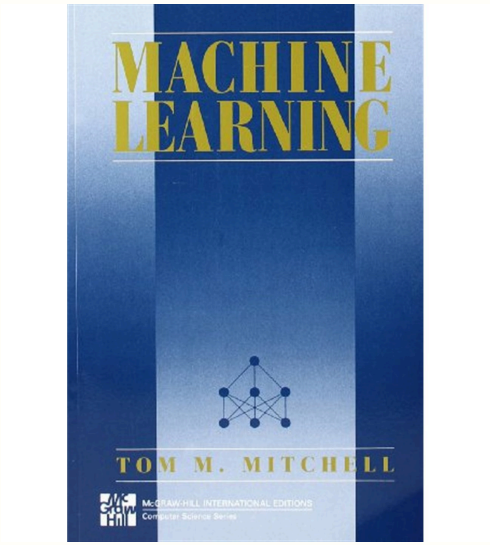
A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training

$$f_T : \mathrm{E} \quad \rightarrow \quad f_A$$

E: Experience

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

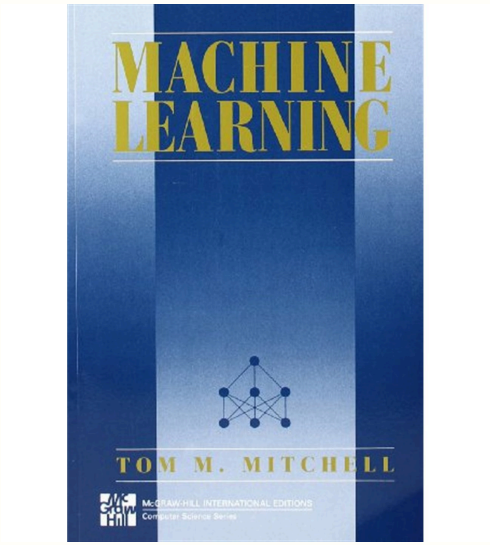
A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training **Task**

$f_T : \mathrm{E} \rightarrow f_A$ $f_A : \mathrm{In} \rightarrow \mathrm{Out}$

E: Experience f_A: Task function

Machine learning defined

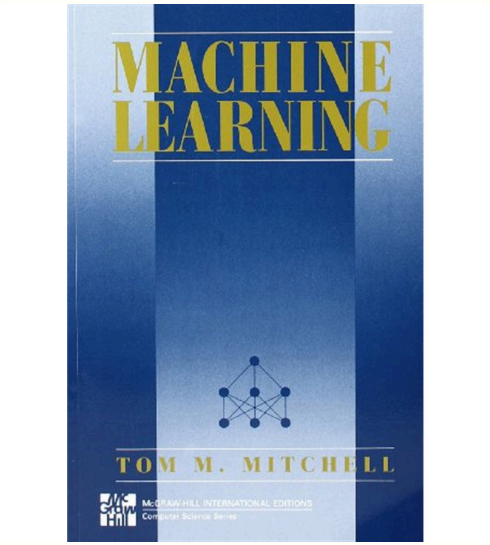


Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training	**Task**	**Evaluation**
$f_T : \mathrm{E} \rightarrow f_A$	$f_A : \mathrm{In} \rightarrow \mathrm{Out}$	$f_P : (\mathrm{In}, \mathrm{Out}) \rightarrow \mathrm{P}$
E: Experience	f_A: Task function	P: Performance

Machine learning defined



Tom M. Mitchell, Machine Learning (1997), cover

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ,

Training	**Task**	**Evaluation**
$f_T : \mathrm{E} \rightarrow f_A$	$f_A : \mathrm{In} \rightarrow \mathrm{Out}$	$f_P : (\mathrm{In}, \mathrm{Out}) \rightarrow \mathrm{P}$
E: Experience	f_A: Task function	P: Performance

- Machine learning is a mapping from $(E, f_P) \rightarrow f_T \rightarrow f_A$
  - That maximizes f_A performance according to P

Supervised experience supports prediction

Experience: paired examples

$$f_T : \mathrm{E} \quad \rightarrow \quad f_A$$

$$f_T : D = \{(x_i, y_i)\}_{i=1}^{N} \quad \rightarrow \quad f_D$$

paired examples → learned predictor

Task: prediction

$$f_A : \mathrm{In} \quad \rightarrow \quad \mathrm{Out}$$

$$f_D : x_* \quad \rightarrow \quad \hat{y}_*$$

new input → predicted output

Supervised experience supports prediction

Experience: paired examples

$$f_T : \mathrm{E} \quad \to \quad f_A$$

$$f_T : D = \{(x_i, y_i)\}_{i=1}^{N} \quad \to \quad f_D$$

paired examples → learned predictor

Task: prediction

$$f_A : \mathrm{In} \quad \to \quad \mathrm{Out}$$

$$f_D : x_* \quad \to \quad \hat{y}_*$$

new input → predicted output

- Experience is a training set $D = \{(x_n, y_n)\}$
  - x: called feature, covariate, or predictor
  - y: called label, target, or response
- Task is to produce a predictor f_D that maps new x_* to correct y_*
- Performance is measured by a loss, introduced shortly

Classification: Iris flowers


Setosa


Versicolor


Virginica

Table 1.1: subset of the Iris design matrix

index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
dots	dots	dots	dots	dots	dots
50	7.0	3.2	4.7	1.4	Versicolor
dots	dots	dots	dots	dots	dots
149	5.9	3.0	5.1	1.8	Virginica

sl: sepal length; sw: sepal width; pl: petal length; pw: petal width

Murphy (2025), Book 1, Fig. 1.1 and Table 1.1

- Classification: $y \in \{1, ..., C\}$
  - The classes are unordered and mutually exclusive

Classification: Iris flowers


Setosa


Versicolor


Virginica

Table 1.1: subset of the Iris design matrix

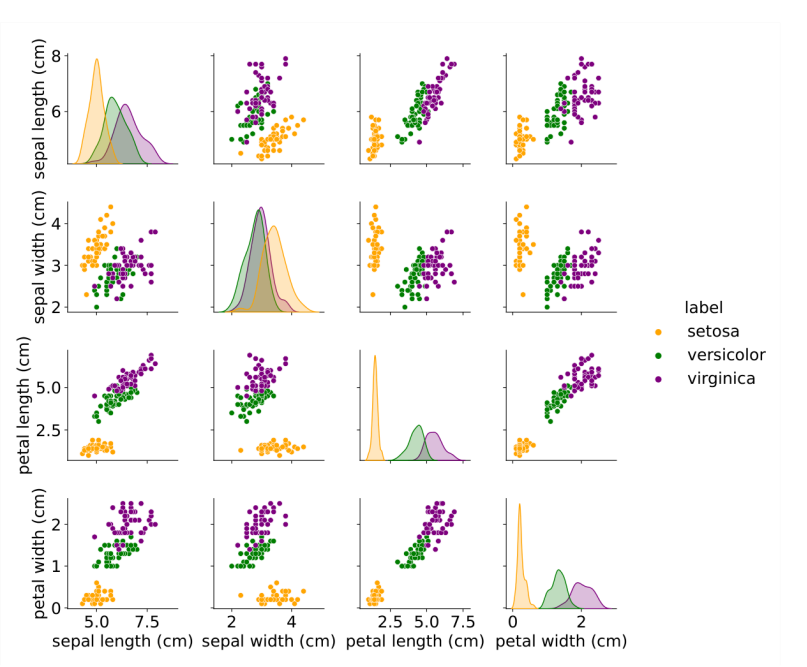
index	sl	sw	pl	pw	label
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
dots	dots	dots	dots	dots	dots
50	7.0	3.2	4.7	1.4	Versicolor
dots	dots	dots	dots	dots	dots
149	5.9	3.0	5.1	1.8	Virginica

sl: sepal length; sw: sepal width; pl: petal length; pw: petal width

Murphy (2025), Book 1, Fig. 1.1 and Table 1.1

- Classification: $y \in \{1, ..., C\}$
  - The classes are unordered and mutually exclusive
- For the Iris dataset
  - $y \in \{\text{Setosa}, \text{Versicolor}, \text{Virginica}\}$
  - $x \in \mathbb{R}^4$

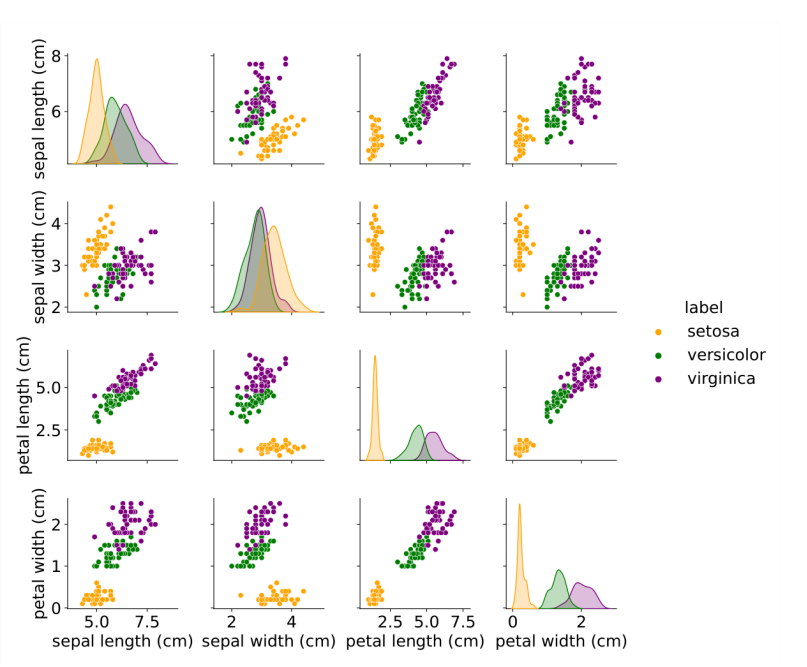
Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class

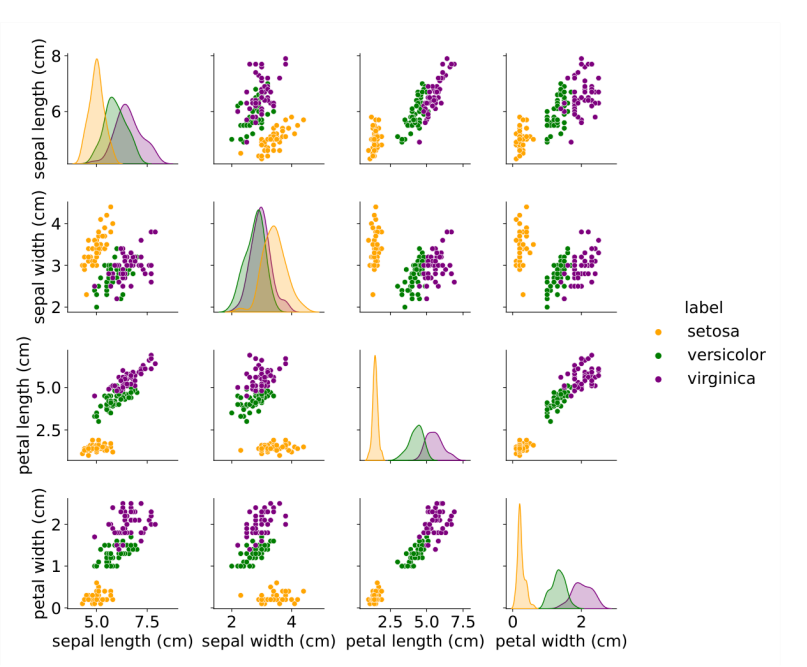
Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class
- Pair plots shows every pair of features
  - Diagonal shows marginal distributions

Exploratory data analysis



Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

- Training set
  - 150 labeled examples
  - 50 in each class
- Pair plots shows every pair of features
  - Diagonal shows marginal distributions
- Always inspect data before modeling
  - Structure
  - Outliers
  - Missing values

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

$$\hat{\theta} = \arg\min_{\theta} L(\theta)$$

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set
- Empirical risk minimization: minimizing the empirical loss

Performance: empirical risk minimization

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} L(x_n, y_n; \theta)$$

$$\hat{\theta} = \arg\min_{\theta} L(\theta)$$

Given samples $D = \{x^1, ..., x^N\}$ from $p(X)$, where $X \in \mathbb{R}$, the empirical distribution is $\hat{p}_{N(x)} = \frac{1}{N} \sum_{n=1}^{N} \delta_{x^n}(x)$.

- The loss function: the cost of predicting $f(x; \theta)$ when the true label is y
- The empirical risk: loss evaluated on the training set
- Empirical risk minimization: minimizing the empirical loss

A loss function

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$\ell(y, f(x; \theta)) = \ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y})$$

- A loss function assigns a cost to a prediction when the true label is known

A loss function

$$L(x, y; \theta) = \ell(y, f(x; \theta))$$

$$\ell(y, f(x; \theta)) = \ell_{01}(y, \hat{y}) = \mathbb{I}(y \neq \hat{y})$$

$$\mathbb{I}(e) = \begin{cases} 1 \text{ if } e \text{ is true} \\ 0 \text{ if } e \text{ is false} \end{cases}$$

- A loss function assigns a cost to a prediction when the true label is known
- The zero-one loss is one for a misclassification and zero for a correct classification
- $\mathbb{I}(e)$ is an indicator function: it evaluates whether condition e holds

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

$$p(y = c \mid x, \theta) \propto p(x \mid y = c, \theta) p(y = c)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions
- Bayes' rule converts those ingredients to posterior class probabilities

A probabilistic perspective

$$p(x \mid y = c, \theta)$$

$$p(y = c \mid x, \theta) \propto p(x \mid y = c, \theta) p(y = c)$$

$$\hat{y} = \arg\max_{c \in \mathcal{Y}} p(y = c \mid x; \theta)$$

- The exploratory data analysis suggests:
  - Features in each class have similar distributions
  - Different classes have different distributions
- Bayes' rule converts those ingredients to posterior class probabilities
- A possible classifier: choose the class with maximum a posteriori probability

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

- **Task:** predict the Iris species from four sepal and petal measurements

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

Experience

What data, feedback, or interaction is available?

$$D = \{(x_n, y_n)\}_{n=1}^{N}$$

150 labeled flowers

- **Task:** predict the Iris species from four sepal and petal measurements
- **Experience:** 150 labeled flowers, with 50 examples from each species

Task, experience, and performance

Task

What must the learned system do?

$$x \in \mathbb{R}^4 \to \hat{y} \in \mathcal{Y}$$

four measurements → species

Experience

What data, feedback, or interaction is available?

$$D = \{(x_n, y_n)\}_{n=1}^{N}$$

150 labeled flowers

Performance

How is success evaluated?

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{I}(y_n \neq f(x_n; \theta))$$

zero-one loss / misclassification rate

- **Task:** predict the Iris species from four sepal and petal measurements
- **Experience:** 150 labeled flowers, with 50 examples from each species
- **Performance:** mean zero-one loss, the training misclassification rate

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs
- Machine learning is a branch of computer science
  - Where the function is learned from experience

Summary: From statistics to machine learning

- Statistics and computer science
  - Statistics focused on the values of unknown parameters
  - Computer science develops functions that map inputs to outputs
- Machine learning is a branch of computer science
  - Where the function is learned from experience
- A learning problem is defined by three ingredients: task, experience, and performance
  - One canonical pairing uses supervised experience $D = \{(x_n, y_n)\}$ for prediction
  - The experience regime and the task are related, but they are not the same taxonomy

Section 3

Classes of machine learning problems

Experience regimes, task classes, and performance criteria

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

Self-supervised

$$D = \{x_n, f(x_n)\} \text{ or } \{f(x_n), x_n\}$$

> Experience describes what the learner gets to observe

Experience taxonomy

Supervised

$$D = \{(x_n, y_n)\}$$

Unsupervised

$$D = \{x_n\}$$

Self-supervised

$$D = \{x_n, f(x_n)\} \text{ or } \{f(x_n), x_n\}$$

Reinforcement / interaction

$$\begin{aligned} \text{decision} &: x(t) \to a(t) \\ \text{action} &: (x(t), a(t)) \to x(t+1) \\ \text{reward} &: x(t+1) \to r(t+1) \end{aligned}$$

> Experience describes what the learner gets to observe

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Prediction

Basic task	Map inputs to outputs
Performance	Successful prediction
Example	Classification or regression

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Generation

Basic task	Produce new examples
Performance	Similarity to the true distribution
Example	Text or image synthesis

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Discovery

Basic task	Find a representation of the data
Performance	Quality of the discovered representation
Example	Clustering or dimensionality reduction

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

Action

Basic task	Choose actions to maximize future reward
Performance	Ability to achieve the desired outcome
Example	Reinforcement learning

Task taxonomy

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

Prediction	Map inputs to outputs
Generation	Produce new examples
Discovery	Find a representation of the data
Action	Choose actions to achieve desired outcomes

There are many combinations of experience and task

Experience	Prediction	Generation	Discovery	Action
Supervised	Classification	—	—	Imitation learning
Unsupervised	—	Density modeling	Clustering / PCA	—
Self-supervised	Next-token prediction	Language generation	Representation learning	—
Reinforcement / interaction	—	—	—	Policy learning

- Not every type of experience supports every task.
- Common pairings do exist.

Summary: Two taxonomies, three ingredients

- **Learning problem** = task + experience + performance
- **Experience regimes:** supervised, unsupervised, self-supervised, interactive
- **Task classes:** prediction, generation, discovery, action
- Probabilistic modeling provides a unifying language

Section 4

From datasets to learning problems

The same dataset can pose different learning problems

Popular datasets

Iris





AGE
TASKS

Since 1936 – about 90 years
species prediction; clustering; visualization

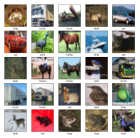
MNIST



AGE
TASKS

Since 1998 – about 28 years
digit classification; representation learning; generation

CIFAR-10



AGE
TASKS

Since 2009 – about 17 years
object classification; robustness; generation

IMDB movie reviews

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

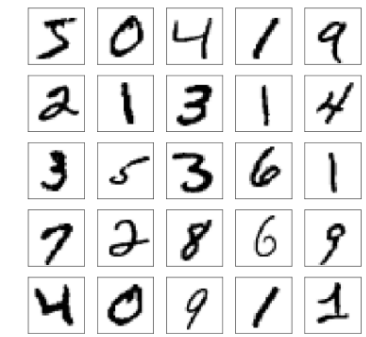
AGE
TASKS

Since 2011 – about 15 years
sentiment classification; text representation

Murphy (2025), Book 1, Figs. 1.1, 1.12a, 1.13b, and Table 1.3

- Machine learning is built around a small number of canonical datasets
  - These four datasets are commonly used
- Each dataset supports multiple tasks
- Datasets support benchmarks that make comparisons easy

MNIST: the same taxonomy, one dataset



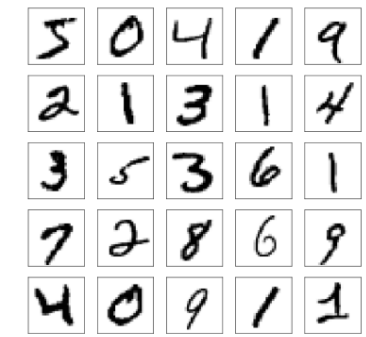
Prediction

$$\hat{y} = f_D(x_*)$$

$$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$$

Murphy (2025), Book 1, Fig. 1.12a

MNIST: the same taxonomy, one dataset



Prediction

$$\hat{y} = f_D(x_*)$$

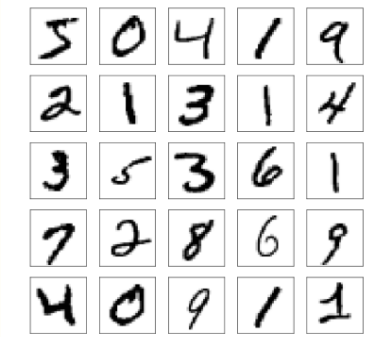
Discovery

$$\hat{z} = f_D(x_*)$$

$$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$$

Murphy (2025), Book 1, Fig. 1.12a

MNIST: the same taxonomy, one dataset



$x_n \in \{0, \ldots, 255\}^{28 \times 28}, \quad y_n \in \{0, \ldots, 9\}$

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

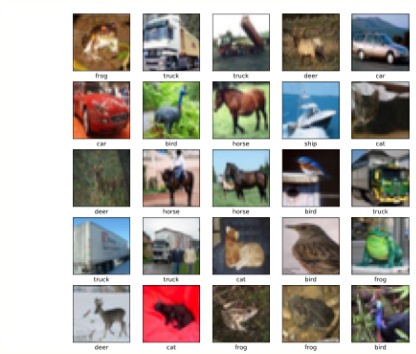
$$\hat{z} = f_D(x_*)$$

Generation

$$x_* \sim p_D(x \mid c)$$

Murphy (2025), Book 1, Fig. 1.12a

CIFAR-10: higher-dimensional inputs, the same questions



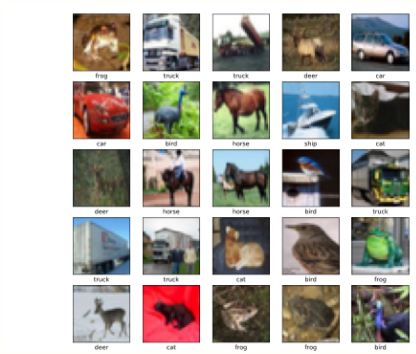
Prediction

$$\hat{y} = f_D(x_*)$$

$$x_n \in \mathbb{R}^{32\times32\times3}, \quad y_n \in \{1, \ldots, 10\}$$

Murphy (2025), Book 1, Fig. 1.13b

CIFAR-10: higher-dimensional inputs, the same questions



Prediction

$$\hat{y} = f_D(x_*)$$

$$L(\theta; D_{\text{train}}) \quad (D_{\text{train}}) \qquad \rightarrow \qquad L(\theta; D_{\text{test}}) \quad (D_{\text{test}})$$

$$x_n \in \mathbb{R}^{32\times32\times3}, \quad y_n \in \{1, \ldots, 10\}$$

Murphy (2025), Book 1, Fig. 1.13b

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Murphy (2025), Book 1, Table 1.3

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

$$\hat{z} = f_D(x_*)$$

Murphy (2025), Book 1, Table 1.3

IMDB reviews: text as sequence data

1. this film was just brilliant casting location scenery story direction everyone's really suited the part they played robert <UNK> is an amazing actor ...
2. big hair big boobs bad music and a giant safety pin these are the words to best describe this terrible movie i love cheesy horror movies and i've seen hundreds...

Prediction

$$\hat{y} = f_D(x_*)$$

Discovery

$$\hat{z} = f_D(x_*)$$

Generation

$$w_t \sim p_D(w_t \mid w_{<t})$$

Murphy (2025), Book 1, Table 1.3

Learning problems become probabilistic queries

Prediction

Task: $\{(x_n, y_n)\} \to \hat{y} = f_D(x_*)$

Performance: $f_P(y_*, \hat{y})$

Generation

Task: $\{x_n\} \to (\{c\}, p_D(x \mid c)) \to x_* \sim p_D(x \mid c)$

Performance: $f_P(p(x \mid c), x_*)$

Discovery

Task: $\{x_n\} \to (\{z\}, f_D(x)) \to \hat{z} = f_D(x_*)$

Performance: $f_P(\hat{z}, x_*)$

Action

Task: $D \to a_* \sim \pi_D(\cdot \mid \{x_{t,*}\}) \to (x_{t+1}, r_{t+1}) = a_t(x_t)$

Performance: $f_P(\{r_t\})$

- Every task invokes probability models: $p_D(y|x)$, $p_D(x|c)$, $p_D(z|x)$
- Performances are expectations over future cases: $f_P = E[\cdot]$
- Probability models provide a common language for all four task schemas

Summary: From datasets to learning problems

- A dataset is not a task
- A learning problem names variables and a query
  - prediction: summarize $p_D(y_* \mid x_*)$
  - discovery: infer $p_D(z_* \mid x_*)$
  - generation: sample from $p_D(x \mid c)$

Section 5

Probability for machine learning

Expectations, moments, and conditioning

Large learning problems still ask familiar probability questions

Observed
x_{obs}: features, measurements, context

Latent
z: state, representation, missing value

Target
y_*: prediction, sample, decision

- The variables may be high-dimensional, latent, or only partly observed

Large learning problems still ask familiar probability questions

Observed
x_{obs}: features, measurements, context

Latent
z: state, representation, missing value

Target
y_*: prediction, sample, decision

Joint distribution: $p_D(x_{\text{obs}}, z, y_*)$

- The variables may be high-dimensional, latent, or only partly observed
- The task determines which quantity we need from the joint distribution

Large learning problems still ask familiar probability questions

Observed
x_{obs}: features, measurements, context

Latent
z: state, representation, missing value

Target
y_*: prediction, sample, decision

Joint distribution: $p_D(x_{\text{obs}}, z, y_*)$

- The variables may be high-dimensional, latent, or only partly observed
- The task determines which quantity we need from the joint distribution
- The probability language is familiar even when the model is large

A few key operations underlie many machine learning tasks

Joint distribution: $p_D(x_{\text{obs}}, z, y_*)$

Summarize

$s(x_{\text{obs}}) = \iint g(y_*) p_D(z, y_* \mid x_{\text{obs}}) \, \mathrm{d}z \, \mathrm{d}y_*$

weighted summary of the target

- Almost all machine learning tasks require an integral
- Latent variables are inferred or averaged out
- Much methodology addresses computing these integrals efficiently

A few key operations underlie many machine learning tasks

Joint distribution: $p_D(x_{\text{obs}}, z, y_*)$

Summarize

$s(x_{\text{obs}}) = \iint g(y_*) p_D(z, y_* \mid x_{\text{obs}}) \, \mathrm{d}z \, \mathrm{d}y_*$

weighted summary of the target

Marginalize

$p_D(y_* \mid x_{\text{obs}}) = \int p_D(z, y_* \mid x_{\text{obs}}) \, \mathrm{d}z$

average out the latent variable

- Almost all machine learning tasks require an integral
- Latent variables are inferred or averaged out
- Much methodology addresses computing these integrals efficiently

A few key operations underlie many machine learning tasks

Joint distribution: $p_D(x_{\text{obs}}, z, y_*)$

Summarize

$s(x_{\text{obs}}) = \iint g(y_*) p_D(z, y_* \mid x_{\text{obs}}) \, \mathrm{d}z \, \mathrm{d}y_*$

weighted summary of the target

Marginalize

$p_D(y_* \mid x_{\text{obs}}) = \int p_D(z, y_* \mid x_{\text{obs}}) \, \mathrm{d}z$

average out the latent variable

Condition

$p_D(z \mid x_{\text{obs}}) = \frac{p_D(x_{\text{obs}}, z)}{\int p_D(x_{\text{obs}}, z') \, \mathrm{d}z'}$

infer the latent variable from observations

- Almost all machine learning tasks require an integral
- Latent variables are inferred or averaged out
- Much methodology addresses computing these integrals efficiently

Generalize the operations using generic random variables

X and Y have joint density $p(x, y)$

Summarize

$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

Marginalize

$$p(x) = \int p(x, y)\,\mathrm{d}y$$

$$p(y) = \int p(x, y)\,\mathrm{d}x$$

Condition

$$p(x \mid y) = \frac{p(x, y)}{p(y)}$$

- The same operations apply to any pair of random variables X and Y
- Lowercase x and y are possible values; conditioning fixes $Y = y$
- The normalizer is another marginal

Expectation is a summarizing integral

Summarize

$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

Continuous

$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

Discrete

$$E[g(X)] = \sum_x g(x)p(x)$$

$$\mu = E[X]$$

Course schematic; Murphy, Book 1 §2.2.5

- Expectation is a probability-weighted average
- Choosing $g(x) = x$ gives the mean

Integrals create linearity

Summarize

$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

$$E[aX + b] = aE[X] + b$$

$$E\left[\sum_i X_i\right] = \sum_i E[X_i]$$

$$E\left[\prod_{i=1}^{n} X_i\right] = \prod_{i=1}^{n} E[X_i] \text{ for independent } X_1, \ldots, X_n$$

- Expectations of sums are sums of expectations
- Independence is not required for sums
- Expectations of products factor when the variables are independent

Variance is another integral

Summarize
$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

Variance
$$V[X] = E\left[(X-\mu)^2\right] = E\left[X^2\right] - \mu^2$$

Second raw moment
$$E\left[X^2\right] = V[X] + \mu^2$$

Course schematic; Murphy, Book 1 §2.2.5

- The second raw moment contains both spread and location

Mean and variance are the first two moments

Summarize
$$E[g(X)] = \int g(x)p(x)\,\mathrm{d}x$$

Raw moments

$$m_1 = E[X]$$
$$m_2 = E[X^2]$$
$$m_k = E[X^k]$$

Central moments

$$\mu_2 = E[(X - E[X])^2] = V[X]$$
$$\mu_k = E[(X - E[X])^k]$$

- A moment is a property of a probability distribution
- Central moments are moments of deviation from the mean

Conditioning is a special case of summarizing

Condition

$$p(x \mid y) = \frac{p(x, y)}{p(y)}$$

$Y = y$
observed information

→

$p(x \mid y)$
$E[X \mid Y = y]$

$$E[X] = E_Y[E[X \mid Y]]$$

- $E[X \mid Y]$ is itself a function of the conditioning variable Y
- Averaging the conditional means recovers the overall mean

Total variance separates within- and between-condition variation

Marginalize

$$p(x) = \int p(x, y)\, \mathrm{d}y$$
$$p(y) = \int p(x, y)\, \mathrm{d}x$$

$$V[X] = \underbrace{E_Y[V[X \mid Y]]}_{\text{within}} + \underbrace{V_Y[E[X \mid Y]]}_{\text{between}}$$

$V[X \mid Y]$
conditional spread

+

$E[X \mid Y]$
changing conditional means

Course schematic; Murphy, Book 1 §2.2.5

Summary: Probability for machine learning

- Expectations and moments summarize a probability distribution
- Conditioning makes those summaries depend on observed information
- Total expectation and total variance combine conditional pieces
- These identities are properties of a distribution treated as given

Section 6

Inverse probability

Estimating unknown quantities from observed data

The goal is to learn from data

Population model

$$Y \sim p(y \mid \theta)$$

unknown θ



Observed sample

$$D = (y_1, \ldots, y_N)$$

What can D tell us about θ ?

- Probability describes possible data given a model
- Statistics uses observed data to learn unknown population quantities

A statistic is something calculated on observed data

$$D \to T(D)$$

Continuous measurements

$$D_x = (x_1, \ldots, x_N)$$
$$\overline{x} = \frac{1}{N}\sum_{n=1}^{N} x_n$$

Bernoulli tosses, Binomial count

$$D_y = (y_1, \ldots, y_N)$$
Each y_n is 0 or 1
$$N_1 = \sum_{n=1}^{N} y_n, \quad N_0 = N - N_1$$

- The sample mean and coin counts are both statistics computed from data
- A statistic's definition must not contain an unknown parameter

An estimator is a statistic aimed at an unknown quantity

$$D \quad \rightarrow \quad \hat{\theta}(D)$$

Target
θ

Estimator
$\hat{\theta}(D)$

- θ will generally be an integral over the distribution as before
- The estimate $\hat{\theta}$ is calculated on the data
  - Different estimates can target the same parameter

Sample moments estimate population moments

Population raw moment

$$m_k = E\left[X^k\right]$$

→

Sample raw moment

$$\hat{m}_k = \frac{1}{N}\sum_{n=1}^{N} x_n^k$$

Population central moment

$$\mu_k = E\left[(X - E[X])^k\right]$$

→

Sample central moment

$$\hat{\mu}_k = \frac{1}{N}\sum_{n=1}^{N} (x_n - \overline{x})^k$$

Section 7

Methods of building estimators

From a sampling model to a fitted parameter

Model fitting chooses an objective

$$D \to L(\theta) \to \hat{\theta} = \arg\min_\theta L(\theta)$$

The data determine an objective; optimizing it produces a point estimate.

Murphy, Book 1 §4.1, Eq. 4.1

- Different estimation methods choose different objectives
- A point estimate alone does not describe uncertainty about its value

Maximum likelihood scores the observed sample

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} p(D \mid \theta)$$

Murphy, Book 1 §4.2.1, Eqs. 4.2–4.3

- Choose the parameter that makes the observed data most likely

Maximum likelihood scores the observed sample

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} p(D \mid \theta)$$

$$p(D \mid \theta) = \prod_{n=1}^{N} p(y_n \mid x_n, \theta)$$

Murphy, Book 1 §4.2.1, Eqs. 4.2–4.3

- Choose the parameter that makes the observed data most likely
- The product assumes independent cases under the same conditional model

Logs turn the likelihood product into a sum

$$\ell(\theta) = \log p(D \mid \theta) = \sum_{n=1}^{N} \log p(y_n \mid x_n, \theta)$$

Murphy, Book 1 §4.2.1, Eqs. 4.4–4.5

- The logarithm preserves the maximizing parameter

Logs turn the likelihood product into a sum

$$\ell(\theta) = \log p(D \mid \theta) = \sum_{n=1}^{N} \log p(y_n \mid x_n, \theta)$$

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} \sum_{n=1}^{N} \log p(y_n \mid x_n, \theta)$$

Murphy, Book 1 §4.2.1, Eqs. 4.4–4.5

- The logarithm preserves the maximizing parameter
- Each case now contributes one additive term

Negative log likelihood gives a minimization objective

$$\mathrm{NLL}(\theta) = -\log p(D \mid \theta) = -\sum_{n=1}^{N} \log p(y_n \mid x_n, \theta)$$

$$\hat{\theta}_{\mathrm{MLE}} = \arg\min_{\theta} \mathrm{NLL}(\theta)$$

Murphy, Book 1 §4.2.1, Eq. 4.6

- Maximizing likelihood and minimizing NLL choose the same parameter
- NLL is the loss form used by many optimization algorithms

The modeled variables determine the likelihood

No given input

$$\hat{\theta}_{\text{MLE}} = \arg\min_\theta - \sum_{n=1}^{N} \log p(y_n \mid \theta)$$

Model inputs and outputs jointly

$$\hat{\theta}_{\text{MLE}} = \arg\min_\theta - \sum_{n=1}^{N} \log p(y_n, x_n \mid \theta)$$

Murphy, Book 1 §4.2.1, Eqs. 4.7–4.8

- The earlier conditional NLL models y_n given x_n
- An unconditional or joint model changes the probability inside the sum

Bernoulli likelihood starts with individual tosses

$$\mathrm{NLL}(\theta) = -\log \prod_{n=1}^{N} p(y_n \mid \theta)$$

Murphy, Book 1 §4.2.3, Eqs. 4.20–4.21

- Reuse the coin model: $y_n = 1$ for heads, $y_n = 0$ for tails

Bernoulli likelihood starts with individual tosses

$$\mathrm{NLL}(\theta) = -\log \prod_{n=1}^{N} p(y_n \mid \theta)$$

$$= -\log \prod_{n=1}^{N} \theta^{I(y_n=1)} (1-\theta)^{I(y_n=0)}$$

Murphy, Book 1 §4.2.3, Eqs. 4.20–4.21

- Reuse the coin model: $y_n = 1$ for heads, $y_n = 0$ for tails
- $I(A)$ equals one when A is true and zero otherwise

The two coin counts summarize the likelihood

$$N_1 = \sum_{n=1}^{N} I(y_n = 1), \quad N_0 = \sum_{n=1}^{N} I(y_n = 0)$$

$$\mathrm{NLL}(\theta) = -[N_1 \log \theta + N_0 \log(1 - \theta)]$$

Murphy, Book 1 §4.2.3, Eq. 4.23

- Once the model is fixed, the order of heads and tails does not affect this likelihood
- $N = N_0 + N_1$ is the sample size

The likelihood estimate is the observed head fraction

$$\frac{d}{d\theta}\,\mathrm{NLL}(\theta) = -\frac{N_1}{\theta} + \frac{N_0}{1-\theta}$$

Murphy, Book 1 §4.2.3, Eqs. 4.24–4.25

- Set the interior derivative to zero and solve for θ

The likelihood estimate is the observed head fraction

$$\frac{d}{d\theta}\,\mathrm{NLL}(\theta) = -\frac{N_1}{\theta} + \frac{N_0}{1-\theta}$$

$$\hat{\theta}_{\mathrm{MLE}} = \frac{N_1}{N_0+N_1} = \frac{N_1}{N}$$

Murphy, Book 1 §4.2.3, Eqs. 4.24–4.25

- Set the interior derivative to zero and solve for θ
- The boundary cases give zero or one when all tosses agree

Gaussian likelihood fits location and spread

$$Y_n \sim \mathcal{N}(\mu, \sigma^2)$$

$$\mathrm{NLL}(\mu, \sigma^2) = -\sum_{n=1}^{N} \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_n-\mu)^2}{2\sigma^2}\right)\right)$$

Murphy, Book 1 §4.2.5, Eqs. 4.32–4.33

- The NLL balances residual size with the proposed variance

Gaussian likelihood fits location and spread

$$Y_n \sim \mathcal{N}(\mu, \sigma^2)$$

$$\mathrm{NLL}(\mu, \sigma^2) = -\sum_{n=1}^{N} \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_n-\mu)^2}{2\sigma^2}\right)\right)$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^{N} (y_n - \mu)^2 + \frac{N}{2} \log(2\pi\sigma^2)$$

Murphy, Book 1 §4.2.5, Eqs. 4.32–4.33

- The NLL balances residual size with the proposed variance

Stationarity gives the Gaussian mean estimate

$$\frac{\partial}{\partial \mu}\ \mathrm{NLL}(\mu, \sigma^2) = 0, \quad \frac{\partial}{\partial \sigma^2}\ \mathrm{NLL}(\mu, \sigma^2) = 0$$

Murphy, Book 1 §4.2.5, Eqs. 4.34–4.35

- The optimum must be stationary in both parameters

Stationarity gives the Gaussian mean estimate

$$\frac{\partial}{\partial \mu}\ \mathrm{NLL}(\mu, \sigma^2) = 0, \quad \frac{\partial}{\partial \sigma^2}\ \mathrm{NLL}(\mu, \sigma^2) = 0$$

$$\hat{\mu}_{\mathrm{MLE}} = \frac{1}{N} \sum_{n=1}^{N} y_n = \overline{y}$$

Murphy, Book 1 §4.2.5, Eqs. 4.34–4.35

- The optimum must be stationary in both parameters
- The fitted Gaussian is centered at the sample mean

Gaussian MLE uses the average squared deviation

$$s^2 = \tfrac{1}{N}\textstyle\sum_{n=1}^{N} y_n^2$$

$$\widehat{\sigma^2}_{\mathrm{MLE}} = \frac{1}{N}\sum_{n=1}^{N}\left(y_n - \hat{\mu}_{\mathrm{MLE}}\right)^2 = \frac{1}{N}\sum_{n=1}^{N}\left(y_n^2 + \hat{\mu}_{\mathrm{MLE}}^2 - 2y_n\hat{\mu}_{\mathrm{MLE}}\right)$$

Murphy, Book 1 §4.2.5, Eqs. 4.36–4.37

- This MLE averages over N observed values

Gaussian MLE uses the average squared deviation

$$s^2 = \tfrac{1}{N}\textstyle\sum_{n=1}^{N} y_n^2$$

$$\widehat{\sigma^2}_{\mathrm{MLE}} = \frac{1}{N}\sum_{n=1}^{N}\left(y_n - \hat{\mu}_{\mathrm{MLE}}\right)^2 = \frac{1}{N}\sum_{n=1}^{N}\left(y_n^2 + \hat{\mu}_{\mathrm{MLE}}^2 - 2y_n\hat{\mu}_{\mathrm{MLE}}\right)$$

$$\widehat{\sigma^2}_{\mathrm{MLE}} = s^2 - \overline{y}^2$$

Murphy, Book 1 §4.2.5, Eqs. 4.36–4.37

- This MLE averages over N observed values
- The first two sample raw moments determine both estimates

Likelihood turns a sampling model into fitted parameters

Bernoulli

$$\hat{\theta}_{\text{MLE}} = \frac{N_1}{N}$$

Gaussian

$$\hat{\mu}_{\text{MLE}} = \bar{y}$$

$$\widehat{\sigma^2}_{\text{MLE}} = s^2 - \bar{y}^2$$

- The model determines the likelihood; the observed sample determines its maximum
- The resulting estimates are functions of the data

Section 8

Empirical risk and moment matching

Two other ways to define a fitting objective

Empirical risk replaces log loss with another loss

$$L(\theta) = \frac{1}{N} \sum_{n=1}^{N} \ell(y_n, \theta; x_n)$$

$$\hat{\theta}_{\text{ERM}} = \arg\min_{\theta} L(\theta)$$

Murphy, Book 1 §4.3, Eq. 4.60

- Empirical risk is the average loss on the observed sample
- Negative log likelihood is one possible loss; the task may motivate another

Method of moments solves matching equations

$$m_k(\theta) = E_\theta[Y^k], \quad \hat{m}_k = \frac{1}{N}\sum_{n=1}^{N} y_n^k$$

Murphy, Book 1 §4.4.1, Eq. 4.68

- Use as many moment equations as parameters to be estimated

Method of moments solves matching equations

$$m_k(\theta) = E_\theta[Y^k], \quad \hat{m}_k = \frac{1}{N}\sum_{n=1}^{N} y_n^k$$

$$m_k(\theta) = \hat{m}_k, \quad k = 1, \ldots, K$$

Murphy, Book 1 §4.4.1, Eq. 4.68

- Use as many moment equations as parameters to be estimated
- Solve for the parameters whose model moments match the sample moments

Gaussian moments recover the likelihood estimates

Match each model moment to its sample counterpart.

$$E[Y] = \mu = \overline{y}$$

$$E\left[Y^2\right] = \sigma^2 + \mu^2 = s^2$$

Murphy, Book 1 §4.4.1.1, Eqs. 4.69–4.70

- Match the first two raw moments to estimate two parameters

Gaussian moments recover the likelihood estimates

Match each model moment to its sample counterpart.

$$E[Y] = \mu = \overline{y}$$

$$E\left[Y^2\right] = \sigma^2 + \mu^2 = s^2$$

$$\hat{\mu}_{\text{MOM}} = \overline{y}, \quad \widehat{\sigma^2}_{\text{MOM}} = s^2 - \overline{y}^2$$

Murphy, Book 1 §4.4.1.1, Eqs. 4.69–4.70

- Match the first two raw moments to estimate two parameters
- For this Gaussian model, MOM and MLE coincide

A uniform model has two unknown endpoints

$$Y \sim \text{Unif}(\theta_1, \theta_2), \quad p(y|\theta) = \frac{1}{\theta_2 - \theta_1} I(\theta_1 \leq y \leq \theta_2)$$

First raw moment

$$m_1 = E[Y] = \frac{\theta_1 + \theta_2}{2}$$

Second raw moment

$$m_2 = E[Y^2] = \frac{\theta_1^2 + \theta_1\theta_2 + \theta_2^2}{3}$$

Murphy, Book 1 §4.4.1.2, Eqs. 4.71–4.73

- Two unknown endpoints require two moment equations

Inverting the moments gives endpoint estimates

$$\theta_1 = m_1 - \sqrt{3(m_2 - m_1^2)}$$

$$\theta_2 = 2m_1 - \theta_1 = m_1 + \sqrt{3(m_2 - m_1^2)}$$

Murphy, Book 1 §4.4.1.2, Eq. 4.74

- The midpoint is the mean; the half-width comes from the variance

Inverting the moments gives endpoint estimates

$$\theta_1 = m_1 - \sqrt{3(m_2 - m_1^2)}$$

$$\theta_2 = 2m_1 - \theta_1 = m_1 + \sqrt{3(m_2 - m_1^2)}$$

Replace m_1, m_2 with the observed sample moments.

Murphy, Book 1 §4.4.1.2, Eq. 4.74

- The midpoint is the mean; the half-width comes from the variance
- An algebraic solution still needs a model check

Moment matching can contradict an observation

$$D = (0, 0, 0, 0, 1), \quad \hat{m}_1 = \hat{m}_2 = 0.2$$

$$\hat{\theta}_1 \approx -0.493, \quad \hat{\theta}_2 \approx 0.893$$

Murphy, Book 1 §4.4.1.2

- The method-of-moments estimate assigns zero probability to an observed value

Moment matching can contradict an observation

$$D = (0, 0, 0, 0, 1), \quad \hat{m}_1 = \hat{m}_2 = 0.2$$

$$\hat{\theta}_1 \approx -0.493, \quad \hat{\theta}_2 \approx 0.893$$

The fitted interval ends at 0.893, but the sample contains 1.

Murphy, Book 1 §4.4.1.2

- The method-of-moments estimate assigns zero probability to an observed value
- Matching selected summaries need not fit the full sample

Uniform likelihood must contain every observation

$$p(D \mid \theta) = (\theta_2 - \theta_1)^{-N} I\left(y_{(1)} \geq \theta_1\right) I\left(y_{(N)} \leq \theta_2\right)$$

Murphy, Book 1 §4.4.1.2, Eqs. 4.75–4.77

- The indicators enforce support for the smallest and largest observations

Uniform likelihood must contain every observation

$$p(D \mid \theta) = (\theta_2 - \theta_1)^{-N} I\left(y_{(1)} \geq \theta_1\right) I\left(y_{(N)} \leq \theta_2\right)$$

$$\Delta = \theta_2 - \theta_1, \quad \frac{d}{d\Delta} \log p(D|\theta) = -\frac{N}{\Delta} < 0$$

Murphy, Book 1 §4.4.1.2, Eqs. 4.75–4.77

- The indicators enforce support for the smallest and largest observations

Uniform likelihood must contain every observation

$$p(D \mid \theta) = (\theta_2 - \theta_1)^{-N} I\left(y_{(1)} \geq \theta_1\right) I\left(y_{(N)} \leq \theta_2\right)$$

$$\Delta = \theta_2 - \theta_1, \quad \frac{d}{d\Delta} \log p(D|\theta) = -\frac{N}{\Delta} < 0$$

$$\hat{\theta}_1 = y_{(1)}, \quad \hat{\theta}_2 = y_{(N)}$$

Murphy, Book 1 §4.4.1.2, Eqs. 4.75–4.77

- The indicators enforce support for the smallest and largest observations
- Among valid intervals, likelihood chooses the narrowest one

The objective determines which data features matter

Moments

Match selected averages to model expectations.

Likelihood

Score the probability of the whole observed sample.

- The Gaussian example gave the same estimates by both routes
- The uniform example exposed a support failure of moment matching

Section 9

MAP and regularization

Point estimation with a prior, then a broader penalty view

A perfect fit to three tosses makes a brittle prediction

$$D = (1, 1, 1), \quad \hat{\theta}_{\mathrm{MLE}} = \tfrac{3}{3} = 1$$

Murphy, Book 1 §4.5

- Three observed heads do not establish that tails are impossible

A perfect fit to three tosses makes a brittle prediction

$$D = (1, 1, 1), \quad \hat{\theta}_{\mathrm{MLE}} = \tfrac{3}{3} = 1$$

$$p\left(y_{\mathrm{next}} = 0 \mid \hat{\theta}_{\mathrm{MLE}}\right) = 0$$

Murphy, Book 1 §4.5

- Three observed heads do not establish that tails are impossible
- This training fit is overconfident about unseen tosses

MAP chooses one value from the posterior

$$p(\theta \mid D) \propto p(D \mid \theta) p(\theta)$$

Murphy, Book 1 §4.5, Eq. 4.92

- The posterior combines the same likelihood with a prior

MAP chooses one value from the posterior

$$p(\theta \mid D) \propto p(D \mid \theta) p(\theta)$$

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} p(\theta \mid D) = \arg\max_{\theta} [\log p(D|\theta) + \log p(\theta)]$$

Murphy, Book 1 §4.5, Eq. 4.92

- The posterior combines the same likelihood with a prior
- MAP reports its mode, so it remains a point estimator

The Beta prior changes the coin objective

$$\theta \sim \mathrm{Beta}(\alpha, \beta), \quad \alpha, \beta > 1$$

$$\ell_{\mathrm{MAP}}(\theta) = \log p(D|\theta) + \log p(\theta)$$

Murphy, Book 1 §4.5.1, Eqs. 4.93–4.94; α, β notation as earlier

- The prior changes the objective before a point estimate is chosen

The Beta prior changes the coin objective

$$\theta \sim \mathrm{Beta}(\alpha, \beta), \quad \alpha, \beta > 1$$

$$\ell_{\mathrm{MAP}}(\theta) = \log p(D|\theta) + \log p(\theta)$$

$$= [N_1 \log \theta + N_0 \log(1-\theta)] + [(\alpha - 1)\log \theta + (\beta - 1)\log(1-\theta)]$$

Murphy, Book 1 §4.5.1, Eqs. 4.93–4.94; α, β notation as earlier

- The prior changes the objective before a point estimate is chosen
- Its two shape parameters contribute to the two log terms

The posterior mode smooths the observed fraction

$$\hat{\theta}_{\text{MAP}} = \frac{N_1 + \alpha - 1}{N_1 + N_0 + \alpha + \beta - 2}$$

Murphy, Book 1 §4.5.1, Eqs. 4.95–4.96

- The Beta$(2, 2)$ prior yields add-one smoothing

The posterior mode smooths the observed fraction

$$\hat{\theta}_{\text{MAP}} = \frac{N_1 + \alpha - 1}{N_1 + N_0 + \alpha + \beta - 2}$$

$$\alpha = \beta = 2 \quad \rightarrow \quad \hat{\theta}_{\text{MAP}} = \frac{N_1 + 1}{N + 2}$$

Murphy, Book 1 §4.5.1, Eqs. 4.95–4.96

- The Beta$(2, 2)$ prior yields add-one smoothing

The posterior mode smooths the observed fraction

$$\hat{\theta}_{\mathrm{MAP}} = \frac{N_1+\alpha-1}{N_1+N_0+\alpha+\beta-2}$$

$$\alpha = \beta = 2 \quad \rightarrow \quad \hat{\theta}_{\mathrm{MAP}} = \frac{N_1+1}{N+2}$$

$$D = (1,1,1): \quad \hat{\theta}_{\mathrm{MLE}} = 1, \quad \hat{\theta}_{\mathrm{MAP}} = \frac{4}{5}$$

Murphy, Book 1 §4.5.1, Eqs. 4.95–4.96

- The Beta$(2,2)$ prior yields add-one smoothing
- The MAP plug-in prediction leaves a nonzero chance of tails

Regularization adds a penalty to empirical risk

$$L(\theta;\lambda) = \left[\frac{1}{N}\sum_{n=1}^{N}\ell(y_n,\theta;x_n)\right] + \lambda C(\theta)$$

$$\lambda \geq 0$$

Murphy, Book 1 §4.5, Eq. 4.89

- Training loss rewards fit to observed cases
- $C(\theta)$ expresses a preference among models; λ sets its strength

A negative log prior is one regularizer

$$C(\theta) = -\log p(\theta)$$

$$L(\theta; \lambda) = -\frac{1}{N} \sum_{n=1}^{N} \log p(y_n | x_n, \theta) - \lambda \log p(\theta)$$

Murphy, Book 1 §4.5, Eqs. 4.90–4.91

- With log loss and matching scaling, minimizing this objective gives MAP

A negative log prior is one regularizer

$$C(\theta) = -\log p(\theta)$$

$$L(\theta; \lambda) = -\tfrac{1}{N} \sum_{n=1}^{N} \log p(y_n | x_n, \theta) - \lambda \log p(\theta)$$

$$N L\big(\theta; \tfrac{1}{N}\big) = -[\log p(D|\theta) + \log p(\theta)]$$

Murphy, Book 1 §4.5, Eqs. 4.90–4.91

- With log loss and matching scaling, minimizing this objective gives MAP
- MAP is one interpretation of regularized fitting

A prior can temper a small sample's extreme fit

Maximum likelihood

Three heads: $\hat{\theta} = 1$

Next-tail probability: 0

MAP with Beta(2, 2)

Three heads: $\hat{\theta} = \frac{4}{5}$

Next-tail probability: $\frac{1}{5}$

- The prior changes a point estimate in a transparent direction
- Its strength matters most when the dataset is small

Section 10

Sampling variation of the MLE

How an estimator changes across repeated datasets

With enough data, an MLE often varies approximately normally

$$\hat{\theta}_{\mathrm{MLE}}\left(\tilde{D}\right) \approx \mathcal{N}\left(\theta^*, \left[NF(\theta^*)\right]^{-1}\right)$$

$$F(\theta) = E_\theta\left[(\nabla_\theta \log p(Y|\theta))(\nabla_\theta \log p(Y|\theta))^T\right]$$

Murphy, Book 1 §4.7.2, Eqs. 4.219–4.220

- θ^* is the fixed true parameter; F is Fisher information
- Repeated datasets produce different estimates around θ^*
- The approximation needs an identifiable, regular model and a large sample

Section 11

Categorical, multinomial, and softmax

The probability language of multiclass labels

Categorical labels

$$\mathrm{Cat}(y \mid \theta) = \prod_{c=1}^{C} \theta_c^{I(y=c)}$$

$y \in \{1, \ldots, C\}$
one label

→

$\theta \in \Delta^{C-1}$
$0 \leq \theta_c \leq 1$
$\sum_c \theta_c = 1$

One-hot encoding

$y = 2$
class index

$\rightarrow$

$\boldsymbol{y} = (0, 1, 0)^T$
one-hot vector

$$\text{Cat}(y \mid \theta) = \prod_{c=1}^{C} \theta_c^{y_c}$$

Multinomial counts

$$\underset{\sum_c N_c = N}{N = (N_1, \ldots, N_C)} \rightarrow \underset{\binom{N}{N_1, \ldots, N_C} \prod_{c=1}^{C} \theta_c^{N_c}}{\mathrm{Mult}(N \mid N, \theta)}$$

Course schematic; Book 1 §2.5

Softmax

$a \in \mathbb{R}^C$
logits



Softmax

$a \in \mathbb{R}^C$

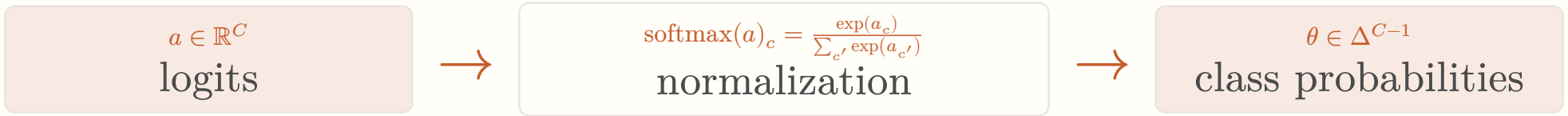
logits



$\text{softmax}(a)_c = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$

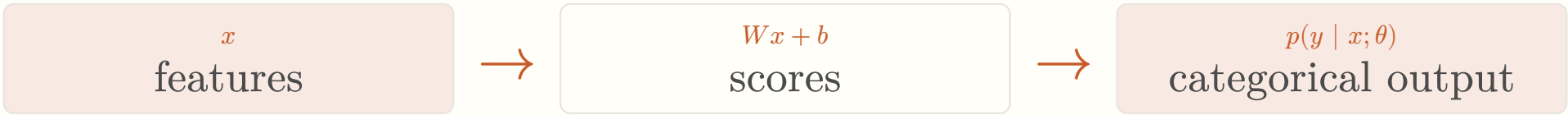
normalization

Softmax


$a \in \mathbb{R}^C$
logits
$\text{softmax}(a)_c = \frac{\exp(a_c)}{\sum_{c'} \exp(a_{c'})}$
normalization
$\theta \in \Delta^{C-1}$
class probabilities

Multiclass logistic notation

$$p(y \mid x; \theta) = \mathrm{Cat}(y \mid \mathrm{softmax}(Wx + b))$$



Summary: Categorical, multinomial, and softmax

- Categorical: one multiclass label
- One-hot vectors encode class membership
- Multinomial: counts from repeated categorical draws
- Softmax maps logits to the probability simplex
- $p(y \mid x; \theta) = \mathrm{Cat}(y \mid \mathrm{softmax}(Wx + b))$

Section 12

Dirac delta and empirical distribution

Observed data as a distribution of point masses

Point mass

$$\lim_{\sigma \to 0} \mathcal{N}(y \mid \mu, \sigma^2) = \delta(y - \mu)$$

$\mathcal{N}(y \mid \mu, \sigma^2)$
σ^2 tends to 0

→

$\delta(y - \mu)$
all mass at μ

Empirical distribution

$$p_{D(y)} = \frac{1}{N} \sum_{n=1}^{N} \delta(y - y_n)$$

$y_1, \ldots, y_N$
observations



p_D
$\frac{1}{N}, \frac{1}{N}, \ldots, \frac{1}{N}$

Course schematic; empirical-distribution definition in the coverage specification

Summary: Empirical distribution

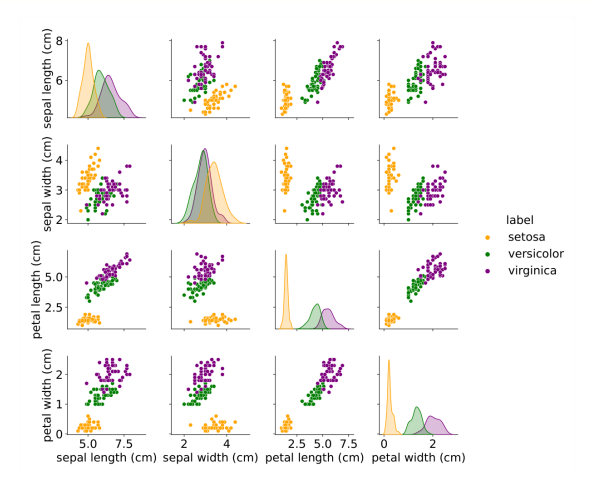
- $\delta(y - \mu)$: a point mass at μ
- $p_{D(y)} = \frac{1}{N} \sum_n \delta(y - y_n)$
- The empirical distribution records observations without smoothing

Section 13

Covariance, correlation, and dependence

Joint variation as geometry

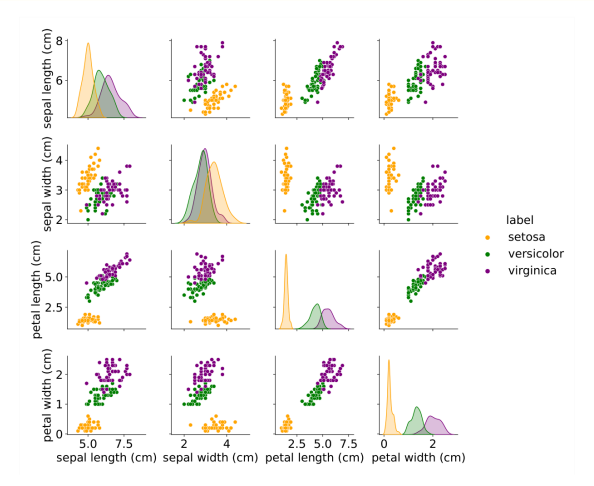
Covariance



$$\mathrm{Cov}[X, Y] = E[(X - E[X])(Y - E[Y])]$$

Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

Covariance



$$\mathrm{Cov}[X, Y] = E[(X - E[X])(Y - E[Y])]$$

> 0	$= 0$	< 0
$+$	0	$-$

Murphy (2025), Book 1, Fig. 1.3; generated by iris_plot.ipynb

Covariance matrix

$$\mathrm{Cov}[\boldsymbol{x}] = E\big[(\boldsymbol{x} - E[\boldsymbol{x}])(\boldsymbol{x} - E[\boldsymbol{x}])^T\big] = \Sigma$$

$$E\big[\boldsymbol{x}\boldsymbol{x}^T\big] = \Sigma + \mu\mu^T$$

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \mathrm{Cov}[X_1, X_2] \\ \mathrm{Cov}[X_2, X_1] & \sigma_2^2 \end{pmatrix}$$

Affine transformations

$\boldsymbol{x}$
random vector



$A\boldsymbol{x} + b$
affine map

$$\mathrm{Cov}[A\boldsymbol{x} + b] = A\ \mathrm{Cov}[\boldsymbol{x}]A^T$$

Correlation

$$\rho = \operatorname{corr}[X, Y] = \frac{\operatorname{Cov}[X,Y]}{\sqrt{V[X]V[Y]}}$$

$\rho \approx 1$	$\rho \approx 0$	$\rho \approx -1$
aligned variation	no linear association	opposed variation

Summary: Covariance and dependence

- Covariance: joint variation around the means
- Σ: covariance matrix
- $E[\boldsymbol{x}\boldsymbol{x}^T] = \Sigma + \mu\mu^T$
- $\mathrm{Cov}[A\boldsymbol{x} + b] = A\Sigma A^T$
- Correlation rescales covariance

Section 14

Multivariate Gaussian and conditioning

Geometry, marginalization, and conditioning

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)\right)$$

$\Sigma = \sigma^2 I$

spherical

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} \, |\Sigma|^{\frac{1}{2}}} \exp\!\left(-\tfrac{1}{2}(y-\mu)^T \Sigma^{-1}(y-\mu)\right)$$

$\Sigma = \sigma^2 I$
spherical

$\Sigma = \mathrm{diag}(\sigma_1^2, \sigma_2^2)$
axis-aligned

Gaussian geometry

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} \, |\Sigma|^{\frac{1}{2}}} \exp\!\left(-\tfrac{1}{2}(y-\mu)^T \Sigma^{-1} (y-\mu)\right)$$

$\Sigma = \sigma^2 I$	$\Sigma = \mathrm{diag}(\sigma_1^2, \sigma_2^2)$	$\Sigma_{12} \neq 0$
spherical	axis-aligned	tilted

Mahalanobis distance

$$\Delta^2 = (y-\mu)^T \Sigma^{-1} (y-\mu)$$

$y - \mu$
displacement

→

Σ^{-1}
geometry-aware scaling

Typical set

$$\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, I_D), \quad R^2 = \|\boldsymbol{x}\|^2 = \sum_{i=1}^{D} x_i^2$$

$E[R^2] = D$
radius squared

,

$\frac{\mathrm{sd}(R^2)}{E[R^2]} = \sqrt{\frac{2}{D}}$
relative width

Gaussian blocks and marginals

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}, \quad \mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}$$

$p(y_1) = \mathcal{N}(y_1 \mid \mu_1, \Sigma_{11})$
first marginal

$p(y_2) = \mathcal{N}(y_2 \mid \mu_2, \Sigma_{22})$
second marginal

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(y_2 - \mu_2)$$

Gaussian conditioning

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_{1|2}, \Sigma_{1|2}\right)$$

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(y_2 - \mu_2)$$

$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

Bivariate conditional

$$p(y_1 \mid y_2) = \mathcal{N}\left(y_1 \mid \mu_1 + \rho\frac{\sigma_1}{\sigma_2}(y_2 - \mu_2), \sigma_1^2(1-\rho^2)\right)$$

$y_2 - \mu_2$
observation

→

$\rho\frac{\sigma_1}{\sigma_2}$
mean shift

→

$\sigma_1^2(1-\rho^2)$
remaining variance

Conditioning as an inference machine

$p(y_1, y_2)$
joint Gaussian

→

y_2
observed

→

$p(y_1 \mid y_2)$
Gaussian posterior

$$\Sigma_{1|2} \leq \Sigma_{11}$$

Summary: Multivariate Gaussian and conditioning

- μ: location; Σ: scale, orientation, and dependence
- Mahalanobis distance follows covariance geometry
- High-dimensional Gaussian mass concentrates in a shell
- Marginals retain covariance blocks
- Conditionals are Gaussian with shifted mean and reduced covariance

Section 15

Missing values as Gaussian conditioning

Inference about hidden entries by conditioning

Visible and hidden entries

$y = (y_v, y_h)$
vector

→

y_v
visible

→

$p(y_h \mid y_v, \theta)$
hidden entries

$$p(y_h \mid y_v, \theta)$$

Summary: Missing values as conditioning

- Partition a vector into visible entries y_v and hidden entries y_h
- Infer hidden entries with $p(y_h \mid y_v, \theta)$
- Gaussian conditioning supplies this distribution analytically

Section 16

Linear Gaussian systems

Latent state, observation, and Gaussian posterior

Linear Gaussian model

$z \sim \mathcal{N}(\mu_z, \Sigma_z)$
latent state

$$p(z, y) = p(z)p(y|z)$$

Linear Gaussian model

$z \sim \mathcal{N}(\mu_z, \Sigma_z)$
latent state



$y \mid z \sim \mathcal{N}(Wz + b, \Sigma_y)$
observation

$$p(z, y) = p(z)p(y|z)$$

Joint Gaussian form

$$\mu = \begin{pmatrix} \mu_z \\ W\mu_z + b \end{pmatrix}$$

$$\Sigma = \begin{pmatrix} \Sigma_z & \Sigma_z W^T \\ W\Sigma_z & \Sigma_y + W\Sigma_z W^T \end{pmatrix}$$

$$\begin{pmatrix} z \\ y \end{pmatrix} \sim \mathcal{N}(\mu, \Sigma)$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

$$\Sigma_{z|y} = \Lambda_{z|y}^{-1}$$

Posterior state

$$p(z \mid y) = \mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Lambda_{z|y} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

$$\Sigma_{z|y} = \Lambda_{z|y}^{-1}$$

$$\mu_{z|y} = \Sigma_{z|y}\left(W^T \Sigma_y^{-1}(y - b) + \Sigma_z^{-1}\mu_z\right)$$

Sensor fusion

Prior
$$\mathcal{N}(z \mid \mu_z, \Sigma_z)$$

$\times$

Measurement
$$\mathcal{N}(y \mid Wz + b, \Sigma_y)$$

$\propto$

Posterior
$$\mathcal{N}\left(z \mid \mu_{z|y}, \Sigma_{z|y}\right)$$

$$\Sigma_{z|y}^{-1} = \Sigma_z^{-1} + W^T \Sigma_y^{-1} W$$

Summary: Linear Gaussian systems

- Prior: $p(z) = \mathcal{N}(z \mid \mu_z, \Sigma_z)$
- Observation: $p(y \mid z) = \mathcal{N}\big(y \mid Wz + b, \Sigma_y\big)$
- Joint model: $p(z, y) = p(z)p(y|z)$
- Posterior: $p(z|y)$ is Gaussian
- Precision adds information from the prior and observation

Lecture 1 summary

- ML tasks specify observed quantities, unknown quantities, and required outputs
- Moments and conditional moments describe probabilistic summaries
- Categorical, multinomial, and softmax distributions describe labels
- Covariance gives multivariate geometry
- Gaussian marginalization and conditioning are reusable inference machinery
- Linear Gaussian systems provide the first complete inference machine